What are Threads?

- **Thread**: a lightweight unit of CPU execution within a process.
- A process can have **one or multiple threads**.
- Example: A web browser → one thread per tab.

Why Do We Need Threads?

1. **Parallelism** – run multiple tasks at once inside one program.
    - Divide a program into n parts → run on n processors.
    - Theoretical speedup = n×, but limited by dependencies & communication.
2. **Avoid Blocking** – prevent waiting on slow I/O.
    - Example: Email client → receive emails + compose emails + notifications simultaneously.

Benefits

- Lower overhead vs creating full processes.
- Shared address space → easy data sharing.
- Faster context switching.
- Supports interactivity + background tasks.

Threads vs. Processes

- **Process = Heavyweight**, **Thread = Lightweight**.
- Threads share memory, files, resources of parent process.
- Easier communication, but less isolation/security.
- Process switching > costly; thread switching < cheaper.

Thread Creation (POSIX)

```
pthread_create(&thread, NULL, function, arg);
pthread_join(thread, NULL);
```

- pthread_create → start a new thread.
- pthread_join → wait until thread finishes.
- Important: If main exits → all threads terminate.

Example Uses

- **Web Server**: dispatcher thread spawns worker threads.
- **Word Processor**: GUI thread + reformatting thread + autosave thread.

Thread Implementation

- **User-Level Threads (ULTs)**: managed in user space. Fast, but blocking one blocks all.
- **Kernel-Level Threads (KLTs)**: managed by OS kernel, more control, slower switches.
- **Models**:
    - Many-to-One → multiple ULTs map to one KLT (efficient but blocking).
    - One-to-One → one ULT per KLT (parallelism, but overhead).
    - Many-to-Many → ULTs mapped dynamically to some KLTs (flexible).

Threading Issues

- **Thread pools** → reuse threads instead of constantly creating/destroying.
- **Thread-specific data** → private storage per thread.
- **Cancellation & Signals** → how to terminate or handle interrupts safely.

CPU Scheduling

Basics

- Needed for **efficient CPU utilization** in multiprogramming.
- **CPU burst** (execution) alternates with **I/O burst** (waiting).
- CPU scheduler = short-term scheduler.
- Decisions occur when process: new, terminates, I/O wait, ready→wait, wait→ready.

Dispatcher

- Gives CPU control to chosen process.

- Functions: context switch, mode switch, program restart.
- **Dispatch latency =** time lost switching.

Scheduling Criteria (Goals)

- **CPU Utilization** → maximize.
- **Throughput** (# processes finished per unit time) → maximize.
- **Turnaround Time** (waiting + execution) → minimize.
- **Waiting Time** (in ready queue) → minimize.
- **Response Time** (first response time) → minimize.

Formulas:

- Turnaround = Waiting + Service
- Waiting = Turnaround - Service

Scheduling Algorithms

1. First-Come, First-Served (FCFS)

- **Non-preemptive**.
- Simple queue (FIFO).
- **Pros**: Fair, easy.
- **Cons**: Convoy effect (short jobs wait for long job).

2. Shortest Job First (SJF)

- Run process with shortest CPU burst.
- **Non-preemptive**: once started, must finish.
- **Preemptive (SRTF)**: shortest *remaining* time first.
- **Pros**: Optimal average waiting time.
- **Cons**: Hard to know burst time, starvation of long jobs.

3. Priority Scheduling

- Highest priority process first.
- Preemptive or non-preemptive.
- **Problem**: Starvation of low-priority jobs.
- **Solution**: Aging (increase priority over time).

4. Round Robin (RR)

- Time quantum q (10–100ms typical).
- Each process gets q CPU, then preempted to end of queue.
- **Pros**: No starvation, good for timesharing.
- **Cons**: Choosing q is tricky:
    - If q too large → FCFS.
    - If q too small → too many context switches (overhead).

Performance Comparisons

- **FCFS**: simple, but convoy effect.
- **SJF**: best for average wait, but unfair to long jobs.
- **Priority**: favors critical jobs, but risks starvation.
- **RR**: balances fairness + responsiveness, depends on q.

Key Exam Tips

- Always draw **Gantt charts** for scheduling problems.
- Calculate **Waiting, Turnaround, Response times** for each process.
- Know when scheduling is **preemptive vs non-preemptive**.
- Understand **trade-offs**: fairness vs efficiency.
- Remember **thread benefits & pitfalls** (parallelism vs blocking/security).